

14.6 - Introduction to Date and Time in Java

Introduction

In Java, data is stored in various data types, which can either be primitive types or classes. For example, the number 10 can be stored using the primitive type `int` or the wrapper class `Integer`. Similarly, a name like "Rohan" is stored using the `String` data type from the `String` class.

```
int i = 10;
```

```
Integer j = 10;
```

```
String name = "Rohan";
```

In the same way, Java provides support for handling **Date and Time**, which are essential components frequently used in applications, especially in databases and transactional operations. These two elements are crucial for tasks like recording timestamps, scheduling events, and managing time-sensitive operations.

Working with Date and Time in Java

Java offers classes for handling date and time, primarily available in two packages:

1. **java.util package:** The `Date` class is provided under this package and can be accessed using `java.util.Date`.
2. **java.sql package:** This package offers `java.sql.Date` for managing date and time, particularly useful in database operations.

Before Java 8, these were the main ways to work with dates. However, since Java 8, a new package called **java.time** was introduced, which provides a modern and more powerful API for date and time handling. This package was developed as part of the **Joda-Time project** by **Joda organisation**, a popular library used prior to Java 8. The `java.time` package in Java 8 replaces the older `java.util.Date` and `java.sql.Date` implementations.

It was also called as **joda.api** or New time and Date API.

👉 Why Use the New java.time Package?

The java.util.Date class has several limitations and design flaws, leading to the need for a better approach to date and time handling:

- The older API has complex and inconsistent methods.
- Many methods in the Date class are deprecated and have been replaced with better alternatives in java.time.
- Working with dates before Java 8 often required using additional libraries like Joda-Time for advanced features.

With the introduction of java.time, the date and time functionalities are now easier to use and offer more precise handling of date and time operations. The java.time package includes classes such as **LocalDate**, **LocalTime**, **LocalDateTime**, and **ZonedDateTime**, which provide a more intuitive way to work with date and time.

👉 Using java.util.Date and java.sql.Date

Before Java 8, java.util.Date and java.sql.Date were commonly used. Here's a basic example to demonstrate how these classes work:

```
// Importing the Date classes
import java.util.Date;
import java.sql.Date;

public class Demo {
    public static void main(String[] args) {
        // Using java.util.Date
        java.util.Date date = new java.util.Date(); // No-
        argument constructor
        System.out.println("Current Date: " + date); //
        Output: Current date and time

        // Getting the time in milliseconds since January 1,
        1970
        long time = date.getTime();
        System.out.println("Milliseconds since epoch: " +
        time); // Output in milliseconds

        // Converting java.util.Date to java.sql.Date
        java.sql.Date sdate = new java.sql.Date(time); //
```

```
Constructor that accepts milliseconds
        System.out.println("SQL Date: " + sdate); // Output in
YYYY-MM-DD format
    }
}
```

Explanation of the Code:

1. Creating a java.util.Date Object:

- We create an instance of java.util.Date using the no-argument constructor, which initializes it with the current date and time.
- The output displays the current date and time in the default format.

2. Getting Time in Milliseconds:

- The getTime() method returns the number of milliseconds since January 1, 1970 (known as the epoch).
- This representation is useful for storing or manipulating date and time values programmatically.

3. Using java.sql.Date:

- We create an instance of java.sql.Date using the time in milliseconds.
- The java.sql.Date class formats the date as YYYY-MM-DD, which is often used in database operations.